

10.06.2026



AĞ TABANLI PARALEL DAĞITIM SİSTEMLERİ DERSİ FİNAL PROJE RAPORU

ZEYNEP LÂL KAŞIKÇI - 22290070

github.com/lalkasikci

İÇİNDEKİLER

1. VERİTABANI PERFORMANS OPTİMİZASYONU VE İZLEME.....	3
1.1 Amaç	3
1.2 Örnek Veritabanının Hazırlanması	3
1.3 İndeks Öncesi Performans Testi	4
1.4 İndeks Yönetimi ve Sorgu İyileştirme	4
1.5 DMV ile İzleme	4
1.6 Disk Alanı, Veri Yoğunluğu ve İndeks Bakımı.....	5
1.7 Rol Tabanlı Erişim Yönetimi	5
3. PROJE 2: VERİTABANI GÜVENLİĞİ VE ERİŞİM KONTROLÜ.....	6
3.1 Amaç	6
3.2 Roller, Kullanıcılar ve Yetkiler.....	6
3.3 Hassas Verilerin Şifrelenmesi	6
3.4 SQL Injection Testleri	7
3.5 Audit Logları	8
4. PROJE 3: VERİTABANI YEDEKLEME VE OTOMASYON ÇALIŞMASI.....	8
4.1 Amaç	8
4.2 Yedekleme Stratejisi.....	8
4.3 SQL Server Agent ile Otomatik Yedekleme	9
4.4 Yedekleme Raporları ve Uyarılar	10
4.5 Eski Yedeklerin Temizlenmesi	10

1. VERİTABANI PERFORMANS OPTİMİZASYONU VE İZLEME

1.1 Amaç

Bu projenin amacı, yüksek sayıda kayıt içeren bir veritabanında sorguların çalışma biçimini incelemek ve uygun optimizasyon yöntemleriyle sorgu maliyetini azaltmaktır. Bu kapsamda örnek bir müşteri-sipariş sistemi kurulmuş, indeks eklenmeden önce ve indeks eklendikten sonra aynı raporlama sorgusu çalıştırılmıştır.

1.2 Örnek Veritabanının Hazırlanması

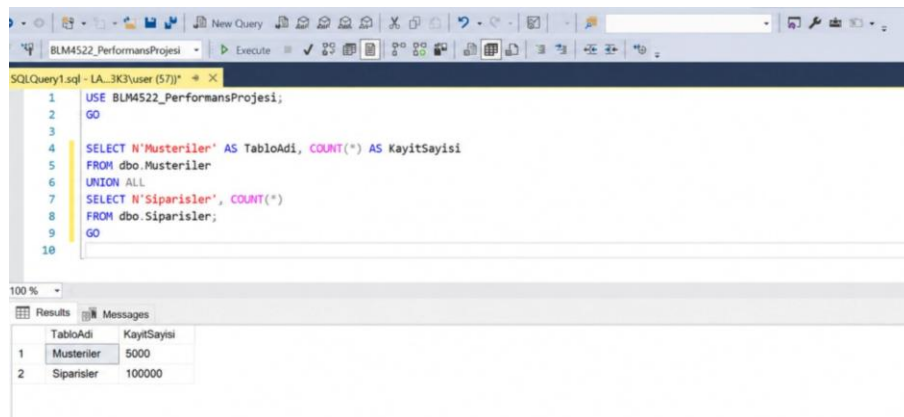
Performans farkını gözlemleyebilmek için BLM4522_PerformansProjesi isimli ayrı bir veritabanı oluşturulmuştur. Veritabanında Musteriler ve Siparisler isimli iki tablo yer almaktadır. Siparisler tablosu, MusteriID alanı üzerinden Musteriler tablosuna foreign key ile bağlanmıştır.

Kod 1. Musteriler ve Siparisler tablolarının oluşturulması

```
CREATE TABLE dbo.Musteriler
(
    MusteriID INT IDENTITY(1,1) PRIMARY KEY,
    AdSoyad NVARCHAR(120) NOT NULL,
    Sehir NVARCHAR(60) NOT NULL,
    KayitTarihi DATE NOT NULL,
    MusteriTipi NVARCHAR(20) NOT NULL
);

CREATE TABLE dbo.Siparisler
(
    SiparisID BIGINT IDENTITY(1,1) PRIMARY KEY,
    MusteriID INT NOT NULL,
    SiparisTarihi DATETIME2(0) NOT NULL,
    ToplamTutar DECIMAL(12,2) NOT NULL,
    SiparisDurumu NVARCHAR(30) NOT NULL,
    CONSTRAINT FK_Siparisler_Musteriler
        FOREIGN KEY (MusteriID) REFERENCES dbo.Musteriler(MusteriID)
);
```

İndeks optimizasyonunun etkisini görünür hale getirmek için otomatik sorgular aracılığıyla 5.000 müşteri ve 100.000 sipariş kaydı üretilmiştir. Siparişler farklı tarihlere, farklı müşterilere ve farklı durumlara dağıtılmıştır.



```
1 USE BLM4522_PerformansProjesi,
2 GO
3
4 SELECT N'Musteriler' AS TabloAdi, COUNT(*) AS KayitSayisi
5 FROM dbo.Musteriler
6 UNION ALL
7 SELECT N'Siparisler', COUNT(*)
8 FROM dbo.Siparisler;
9 GO
10
```

TabloAdi	KayitSayisi
Musteriler	5000
Siparisler	100000

Şekil 1. Musteriler ve Siparisler tablolarındaki kayıt sayılarının SSMS çıktısı

1.3 İndeks Öncesi Performans Testi

İlk ölçüm aşamasında Siparisler tablosunda filtreleme alanlarını kapsayan yardımcı bir indeks bulunmamaktadır. SSMS üzerinde Actual Execution Plan özelliği Ctrl + M kısayolu ile açılmış; STATISTICS IO ve STATISTICS TIME seçenekleri aktif edilmiştir. Böylece sorgunun okuduğu veri sayfaları ve çalışma süresi incelenmiştir.

Kod 2. İndeks öncesi performans sorgusu

```
SET STATISTICS IO ON;
SET STATISTICS TIME ON;

SELECT
    s.MusteriID,
    m.AdSoyad,
    COUNT(*) AS SiparisSayisi,
    SUM(s.ToplamTutar) AS ToplamHarcama
FROM dbo.Siparisler AS s
INNER JOIN dbo.Musteriler AS m
    ON m.MusteriID = s.MusteriID
WHERE s.SiparisDurumu = N'Tamamlandı'
    AND s.SiparisTarihi >= DATEADD(MONTH, -12, SYSDATETIME())
GROUP BY s.MusteriID, m.AdSoyad
ORDER BY ToplamHarcama DESC;
```

Sorgu, son 12 ay içinde tamamlanan siparişleri müşteri bazında gruplayarak sipariş sayısını ve toplam harcama tutarını hesaplamaktadır. İndeks bulunmadığı durumda SQL Server, filtrelenen satırlara ulaşmak için daha geniş kapsamlı bir tarama gerçekleştirebilir.

1.4 İndeks Yönetimi ve Sorgu İyileştirme

İlk testin ardından sorguda filtreleme amacıyla kullanılan SiparisDurumu ve SiparisTarihi sütunlarını kapsayan bir nonclustered indeks oluşturulmuştur. MusteriID ve ToplamTutar sütunları ise INCLUDE bölümüne eklenmiştir. Bu yaklaşım, sorgunun ihtiyaç duyduğu ek verilerin indeks üzerinden okunabilmesini ve ana tabloya dönüş ihtiyacının azaltılmasını hedeflemektedir.

Kod 3. Performans iyileştirmesi için oluşturulan indeksler

```
CREATE NONCLUSTERED INDEX IX_Siparisler_Durum_Tarih
ON dbo.Siparisler (SiparisDurumu, SiparisTarihi)
INCLUDE (MusteriID, ToplamTutar);

CREATE NONCLUSTERED INDEX IX_Siparisler_MusteriID
ON dbo.Siparisler (MusteriID)
INCLUDE (SiparisTarihi, ToplamTutar, SiparisDurumu);
```

İndeksler oluşturulduktan sonra indeks öncesinde kullanılan sorgunun aynısı yeniden çalıştırılmıştır. Karşılaştırmanın doğru yapılabilmesi için sorgu metni değiştirilmemiştir. İndeks sonrası ölçümde logical reads, CPU time, elapsed time ve Execution Plan sonuçları tekrar incelenmiştir.

1.5 DMV ile İzleme

Dynamic Management View (DMV) yapıları, SQL Server'ın çalışma durumu hakkında bilgi veren sistem görünümleridir. Projede maliyetli sorguların tespit edilmesi, eksik indeks önerilerinin görüntülenmesi, mevcut indekslerin kullanım durumlarının izlenmesi ve aktif sorguların kontrol edilmesi amacıyla DMV sorguları kullanılmıştır.

1.6 Disk Alanı, Veri Yoğunluğu ve İndeks Bakımı

Performans yönetimi yalnızca sorgu sürelerinin azaltılmasından ibaret değildir. Veritabanı dosyalarının doluluk oranı, tabloların disk alanı kullanımı ve indeks parçalanması da izlenmelidir. sys.database_files görünümü ile veri ve log dosyalarının toplam boyutu, kullanılan alanı ve boş alanı görüntülenmiştir.

Kod 5. Disk alanı analiz sorgusu

```
SELECT
    name AS DosyaAdi,
    type_desc AS DosyaTuru,
    physical_name AS FizikselKonum,
    CAST(size / 128.0 AS DECIMAL(18,2)) AS ToplamBoyut_MB,
    CAST(FILEPROPERTY(name, 'SpaceUsed') / 128.0 AS DECIMAL(18,2))
    AS KullanilanAlan_MB
FROM sys.database_files;
```

İndeks fragmentasyonu sys.dm_db_index_physical_stats görünümü ile incelenmiştir. Örnek bakım işlemi olarak IX_Siparisler_Durum_Tarih indeksi üzerinde REORGANIZE komutu uygulanmıştır.

Kod 6. Örnek indeks bakım işlemi

```
ALTER INDEX IX_Siparisler_Durum_Tarih
ON dbo.Siparisler
REORGANIZE;
```

1.7 Rol Tabanlı Erişim Yönetimi

Proje kapsamındaki veri yöneticisi rolleri gereksinimini karşılamak amacıyla yalnızca okuma yetkisine sahip bir rol ile veri düzenleme yetkisine sahip bir yönetici rolü oluşturulmuştur. Demo kullanıcıları WITHOUT LOGIN yöntemiyle tanımlanarak sunucu seviyesinde yeni bir giriş hesabı oluşturmadan yetki testleri gerçekleştirilmiştir.

Kod 7. Performans projesinde rol tabanlı yetkilendirme

```
CREATE ROLE db_performans_okuyucu;
CREATE ROLE db_performans_yonetici;

GRANT SELECT ON dbo.Musteriler TO db_performans_okuyucu;
GRANT SELECT ON dbo.Siparisler TO db_performans_okuyucu;

GRANT SELECT, INSERT, UPDATE, DELETE
ON dbo.Siparisler TO db_performans_yonetici;
```

Okuyucu rolünün veri görüntüleyebildiği ancak DELETE işlemi yapamadığı iki ayrı test ile doğrulanmıştır. Böylece en az yetki ilkesi uygulanmıştır.



Şekil 2: Yetkisiz Silme İşlemi

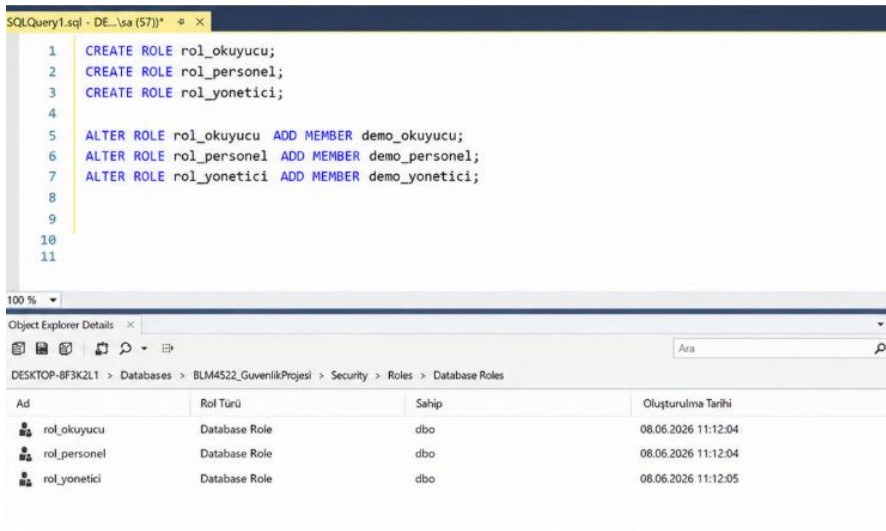
2. VERİTABANI GÜVENLİĞİ VE ERIŞİM KONTROLÜ

2.1 Amaç

Bu projenin amacı, MSSQL üzerinde kullanıcıların yalnızca görevleri için gerekli işlemleri yapabilmesini sağlamak, hassas verileri şifrelemek, SQL injection riskini göstermek ve kullanıcı hareketlerini denetlenebilir hale getirmektir. Uygulamada BLM4522_GuvenlikProjesi isimli örnek bir veritabanı ve Musteriler tablosu oluşturulmuştur.

2.2 Roller, Kullanıcılar ve Yetkiler

Erişim yönetimi için rol_okuyucu, rol_personel ve rol_yonetici isimli üç farklı rol tanımlanmıştır. Okuyucu rolüne yalnızca SELECT, personel rolüne SELECT ve UPDATE, yönetici rolüne ise SELECT, INSERT, UPDATE ve DELETE yetkileri verilmiştir.



Şekil 3: Okuyucu, personel ve yönetici rollerinin SSMS üzerinde gösterilmesi

WITHOUT LOGIN yöntemiyle oluşturulan demo kullanıcıları ilgili rollere üye yapılmıştır. Ayrıca isteğe bağlı sorgularla SQL Server Authentication ve Windows Authentication örnekleri hazırlanmıştır. SQL Server Authentication için sunucu seviyesinde bir login ve parola tanımlanabilir.

2.3 Hassas Verilerin Şifrelenmesi

Müşteri tablosunda hassas verilerin açık metin olarak saklanmasını önlemek amacıyla sütun bazlı şifreleme uygulanmıştır. Önce veritabanı master key'i, ardından sertifika ve AES_256 algoritmasını kullanan symmetric key oluşturulmuştur.

Kod 9. Sütun bazlı şifreleme için anahtar ve sertifika oluşturulması

```
CREATE MASTER KEY
ENCRYPTION BY PASSWORD = 'GucluDemoParola_2026!';

CREATE CERTIFICATE MusteriVeriSertifikasi
WITH SUBJECT = N'Müşteri hassas verilerini koruma sertifikası';
```

```
CREATE SYMMETRIC KEY MusteriVeriAnahtari  
WITH ALGORITHM = AES_256  
ENCRYPTION BY CERTIFICATE MusteriVeriSertifikasi;
```

Şifreli alan VARBINARY veri tipinde saklanmıştır. Veriyi görüntülemek için symmetric key açılmış ve DECRYPTBYKEY fonksiyonu kullanılmıştır. Anahtar açılmadan gerçekleştirilen sorgularda hassas veri okunabilir metin olarak görüntülenmemektedir.

Kod 10. Şifreli verinin kontrollü olarak çözülmesi

```
OPEN SYMMETRIC KEY MusteriVeriAnahtari  
DECRYPTION BY CERTIFICATE MusteriVeriSertifikasi;  
  
SELECT  
    MusteriID,  
    AdSoyad,  
    CONVERT(NVARCHAR(20), DECRYPTBYKEY(TCNoSifreli)) AS CozulmusTCNo  
FROM dbo.Musteriler;  
  
CLOSE SYMMETRIC KEY MusteriVeriAnahtari;
```

2.4 SQL Injection Testleri

SQL injection, kullanıcı girdisinin doğrudan sorgu metnine eklenmesi sonucunda sorgu yapısının değiştirilmesi riskidir. Projede kontrollü bir eğitim senaryosu hazırlanmış ve yalnızca SELECT işlemi gerçekleştiren zararsız bir örnek kullanılmıştır.

Kod 11. Kullanıcı girdisinin doğrudan sorguya eklenmesinden kaynaklanan risk

```
DECLARE @SupheliGirdi NVARCHAR(200) = N''' OR 1=1 --';  
DECLARE @RiskliSorgu NVARCHAR(MAX);  
  
SET @RiskliSorgu =  
    N'SELECT MusteriID, AdSoyad, Sehir  
    FROM dbo.Musteriler  
    WHERE AdSoyad = N''' + @SupheliGirdi + N''';  
  
EXEC (@RiskliSorgu);
```

Riskli yaklaşımda şüpheli girdi WHERE koşulunu etkisiz hale getirebilir. Güvenli yaklaşımda ise sp_executesql ile parametrelili sorgu kullanılmıştır. Parametrelili sorgu, kullanıcı girdisini SQL komutunun bir parçası olarak değil veri olarak işler.

Kod 12. SQL injection riskini azaltan parametrelili sorgu

```
DECLARE @KullaniciGirdisi NVARCHAR(200) = N''' OR 1=1 --';  
DECLARE @GüvenliSorgu NVARCHAR(MAX);  
  
SET @GüvenliSorgu =  
    N'SELECT MusteriID, AdSoyad, Sehir  
    FROM dbo.Musteriler  
    WHERE AdSoyad = @AdSoyad';  
  
EXEC sp_executesql  
    @GüvenliSorgu,  
    N'@AdSoyad NVARCHAR(200)',  
    @AdSoyad = @KullaniciGirdisi;
```

2.5 Audit Logları

Kullanıcı aktivitelerinin izlenebilmesi için AuditLog isimli bir tablo oluşturulmuştur. Musteriler tablosu üzerinde INSERT, UPDATE veya DELETE işlemi gerçekleştirildiğinde trigger çalışmakta ve işlem türü, kullanıcı adı, işlem zamanı ve etkilenen kayıt numarası log tablosuna eklenmektedir.

Kod 13. Audit log tablosu

```
CREATE TABLE dbo.AuditLog
(
    AuditID    BIGINT IDENTITY(1,1) PRIMARY KEY,
    TabloAdi   SYSNAME NOT NULL,
    IslemTuru  NVARCHAR(10) NOT NULL,
    KullaniciAdi SYSNAME NOT NULL,
    IslemZamani DATETIME2(0) NOT NULL DEFAULT SYSDATETIME(),
    EtkilenenKayit INT NULL,
    Aciklama   NVARCHAR(500) NULL
);
```

Trigger tabanlı çözüm, farklı SQL Server sürümlerinde uygulanabilir bir denetim mekanizması sunmaktadır. Ayrıca SQL Server sürümü desteklediğinde yerleşik SQL Server Audit özelliği için ayrı bir örnek sorgu hazırlanmıştır. Bu sorgu, audit kayıtlarını dosyaya yazacak şekilde yapılandırılabilir.

3. VERİTABANI YEDEKLEME VE OTOMASYON ÇALIŞMASI

3.1 Amaç

Bu projenin amacı, veritabanı yedekleme işlemlerini manuel bir görev olmaktan çıkararak zamanlanmış ve izlenebilir bir sürece dönüştürmektir. BLM4522_OtomasyonProjesi isimli örnek veritabanı için tam yedek, fark yedeği ve transaction log yedeği işlemleri ayrı prosedürler halinde hazırlanmıştır. SQL Server Agent job'ları bu prosedürleri belirlenen zamanlarda otomatik olarak çalıştırmaktadır.

3.2 Yedekleme Stratejisi

Transaction log yedeğinin alınabilmesi için veritabanı FULL recovery model kullanacak şekilde yapılandırılmıştır. Tam yedek, veritabanının tamamını; fark yedeği, son tam yedekten itibaren değişen verileri; transaction log yedeği ise gerçekleşen işlemlerin kayıtlarını içerir.

Kod 14. FULL recovery model ayarı

```
ALTER DATABASE BLM4522_OtomasyonProjesi
SET RECOVERY FULL;

SELECT
    name AS VeritabaniAdi,
    recovery_model_desc AS RecoveryModel
FROM sys.databases
WHERE name = 'BLM4522_OtomasyonProjesi';
```

Yedekleme prosedürlerinde BACKUP DATABASE veya BACKUP LOG komutları dinamik dosya adlarıyla çalıştırılmaktadır. Dosya adlarına tarih ve saat eklenerek her işlem için ayrı bir yedek dosyası oluşturulmaktadır.

Kod 15. Tam yedek alma işleminin temel yapısı

```
BACKUP DATABASE BLM4522_OtomasyonProjesi  
TO DISK = N'C:\SQLBackup\BLM4522_OtomasyonProjesi_FULL_...bak'  
WITH INIT, COMPRESSION, CHECKSUM, STATS = 10;
```

Ad	Değiştirilme tarihi	Tür	Boyut
BLM4522_OtomasyonProjesi_FULL_20260608_020000.bak	08.06.2026 02:00	BAK File	1.245.760 KB
BLM4522_OtomasyonProjesi_DIFF_20260608_080000.bak	08.06.2026 08:00	BAK File	256.512 KB
BLM4522_OtomasyonProjesi_DIFF_20260608_140000.bak	08.06.2026 14:00	BAK File	248.320 KB
BLM4522_OtomasyonProjesi_LOG_20260608_090000.trn	08.06.2026 09:00	TRN File	16.384 KB
BLM4522_OtomasyonProjesi_LOG_20260608_100000.trn	08.06.2026 10:00	TRN File	16.384 KB
BLM4522_OtomasyonProjesi_LOG_20260608_110000.trn	08.06.2026 11:00	TRN File	16.384 KB
BLM4522_OtomasyonProjesi_LOG_20260608_120000.trn	08.06.2026 12:00	TRN File	16.384 KB
BLM4522_OtomasyonProjesi_LOG_20260608_130000.trn	08.06.2026 13:00	TRN File	16.384 KB

Şekil 4: Oluşan tam, fark ve log yedekleri

3.3 SQL Server Agent ile Otomatik Yedekleme

SQL Server Agent kullanılarak üç temel yedekleme job'ı oluşturulmuştur. Günlük tam yedek işi her gün saat 02:00'de, fark yedeği işi 6 saatte bir, transaction log yedeği işi ise saatte bir çalışacak şekilde zamanlanmıştır. Bunun yanında yedeklerin zamanında alınıp alınmadığını kontrol eden günlük bir job ve eski dosyaları temizleyen haftalık bir bakım job'ı da hazırlanmıştır.

Kod 16. SQL Server Agent job oluşturma örneği

```
EXEC msdb.dbo.sp_add_job  
  @job_name = N'BLM4522 - Günlük Tam Yedek',  
  @enabled = 1;  
  
EXEC msdb.dbo.sp_add_jobstep  
  @job_name = N'BLM4522 - Günlük Tam Yedek',  
  @step_name = N'Tam yedek prosedürünü çalıştır',  
  @subsystem = N'TSQL',  
  @database_name = N'BLM4522_OtomasyonProjesi',  
  @command = N'EXEC dbo.usp_TamYedekAl';
```

sp_add_schedule ile job çalışma zamanı belirlenmiş, sp_attach_schedule ile ilgili job'a bağlanmış ve sp_add_jobserver ile yerel sunucuda aktif hale getirilmiştir. Bu yapı sayesinde insan müdahalesi olmadan düzenli yedekleme yapılabilmektedir.

3.4 Yedekleme Raporları ve Uyarılar

Yedekleme işlemlerinin yalnızca zamanlanması yeterli değildir. Düzenli çalışıp çalışmadıklarının izlenmesi gerekir. Bu amaçla YedeklemeOtomasyonLogu isimli bir tablo oluşturulmuştur. Prosedürler TRY-CATCH blokları içerisinde çalışmakta; başarılı veya başarısız işlemler dosya yolu, başlangıç zamanı, bitiş zamanı ve hata mesajı ile birlikte tabloya kaydedilmektedir.

Kod 17. Yedekleme otomasyon log tablosu

```
CREATE TABLE dbo.YedeklemeOtomasyonLogu
(
    LogID          BIGINT IDENTITY(1,1) PRIMARY KEY,
    VeritabaniAdi  SYSNAME NOT NULL,
    YedekTuru      NVARCHAR(20) NOT NULL,
    DosyaYolu      NVARCHAR(500) NULL,
    BasariliMi     BIT NOT NULL,
    IslemBaslangic DATETIME2(0) NOT NULL,
    IslemBitis     DATETIME2(0) NOT NULL,
    HataMesaji     NVARCHAR(2000) NULL
);
```

msdb.dbo.backupset ve msdb.dbo.backupmediafamily tabloları üzerinden yedek geçmişi raporu hazırlanmıştır. Rapor; yedek türünü, başlangıç ve bitiş zamanlarını, dosya boyutunu ve fiziksel dosya yolunu göstermektedir. Ayrıca PowerShell ile aynı bilgileri CSV dosyasına aktaran alternatif bir raporlama scripti hazırlanmıştır.

3.5 Eski Yedeklerin Temizlenmesi

Yedek dosyalarının sürekli birikmesi disk alanının dolmasına neden olabilir. Bu nedenle 14 günden eski .bak ve .trn uzantılı dosyaları temizleyen bir bakım prosedürü hazırlanmıştır. SQL Server Agent job'ı bu prosedürü haftalık olarak çalıştırmaktadır.

Kod 18. Süresi dolan yedek dosyalarının temizlenmesi

```
EXEC master.dbo.xp_delete_file
0,
N'C:\SQLBackup\BLM4522',
N'bak',
@SilmeSiniri,
1;

EXEC master.dbo.xp_delete_file
0,
N'C:\SQLBackup\BLM4522',
N'trn',
@SilmeSiniri,
1;
```

Kontrollü hata testi amacıyla geçersiz bir dosya yolu kullanılarak başarısız yedekleme senaryosu hazırlanmıştır. TRY-CATCH yapısı, oluşan hatayı YedeklemeOtomasyonLogu tablosuna kaydetmektedir. Böylece sistemin hata durumlarında nasıl davranacağı gösterilebilmektedir.

Video : https://drive.google.com/file/d/1G3biAUj_E0gd7cUJp3jwkwyztncT4aVC/view?usp=sharing